

2万件のパイプラインを データ需要の行動ログとして読み解く

ODMに存在する約20,000件のデータパイプライン（XML）を解析し、銀行内で繰り返し発生してきたデータ需要と加工パターンを構造として復元する。

目的は「2万本を移植すること」ではなく、複数部門・複数用途から再利用できる共通データ資産の候補を、コードの利用実態から発見することである。

Data Usage Graph

Connected Table Set / 第N連結

Pipeline Fingerprint

Generic Mart Candidate

Databricks / Unity Catalog / Delta

2万件のパイプラインは、移行対象ではなく需要の記録である

同じ資産を「何を移すか」で見るか「何が求められてきたか」で見るかで、得られる結論が変わる。

従来の見方：移行対象の山

パイプライン1本ずつを個別資産として数え、移植可否・工数・依存関係を積み上げる。

結果として得られるのは「**2万本の作業リスト**」であり、重複した加工や重複した中間データはそのまま新環境へ持ち込まれる。

重複が温存される

今回の見方：データ需要の行動ログ

1本のパイプラインを「**誰が・どのデータを組み合わせ・どう加工し・何を作ろうとしたか**」の記録として読む。

2万本を横断して集計すれば、**繰り返し発生している共通のデータ需要**が統計的に浮かび上がる。

共通資産として集約できる

個別ユーザー・個別部門で重複して作られている中間データと加工ロジックを発見し、複数ユースケースから再利用可能な汎用データマート候補を抽出する。

「汎用データマート」をどう定義するか

複数の既存データパイプラインで共通して利用・生成されているデータ、加工ロジック、業務定義を標準化し、特定の帳票・分析・ユーザーに依存せず、複数の分析・AI・業務ユースケースから再利用可能にしたデータセット。

依存しない

特定の帳票、特定の分析、特定の作成者に紐づかない。用途が消えても価値が残る単位で切る。

再利用される

複数部門・複数Output・複数年にわたって参照される。単発需要と恒常需要を明確に分ける。

標準化されている

データ構造だけでなく、JOINキー・粒度・集計定義まで揃っていて初めて共通資産になる。

重要：「よく使われる既存テーブル」をそのまま汎用化することではない

今回重視するのは、InputからOutputへ至る途中で、複数Pipelineが繰り返し同じテーブル群をJOIN・加工して作っている「共通中間層」を見つけることである。

単一テーブルの人気度から、連結テーブル群の共通性へ

従来の単純な考え方

よく使われるテーブル

↓

汎用化候補

CUSTOMER が 1,500 Pipeline で使われる という事実だけでは、何を作るべきかまでは決まらない。CUSTOMERは既に存在しており、作り直す対象ではない。

今回の考え方

複数Pipelineで同じテーブル群が繰り返し連結される

↓ 第N連結ごとに共通性を見る

↓ どこまで共通利用が維持されるかを見る

↓ 次の連結で利用数が急落する境界を見る

↓ 共通中間層候補

↓ Fingerprintで加工ロジックも比較

↓ 汎用データマート候補

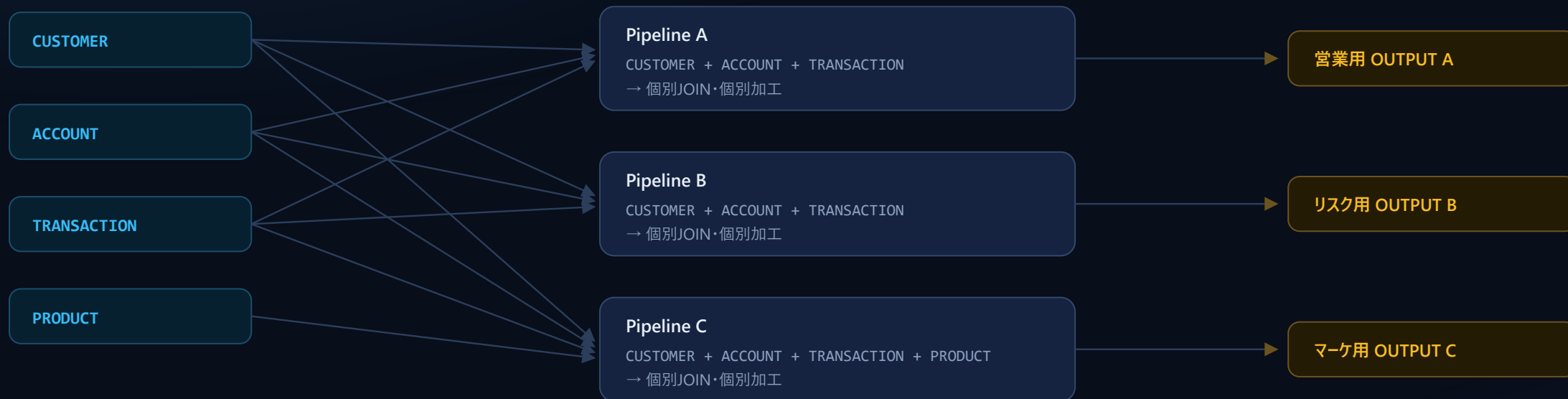
探すのは点（単一テーブル）ではなく、面（セットで連結されたテーブル群）。

「よく読まれるテーブル」ではなく、「毎回同じように組み立て直されている中間結果」こそが、一度作って共有すべき資産である。

現状：同じ中間データが Pipeline ごとに作り直されている

複数の部門が、それぞれのOutputのために、毎回ゼロから同じテーブル群をJOINしている。

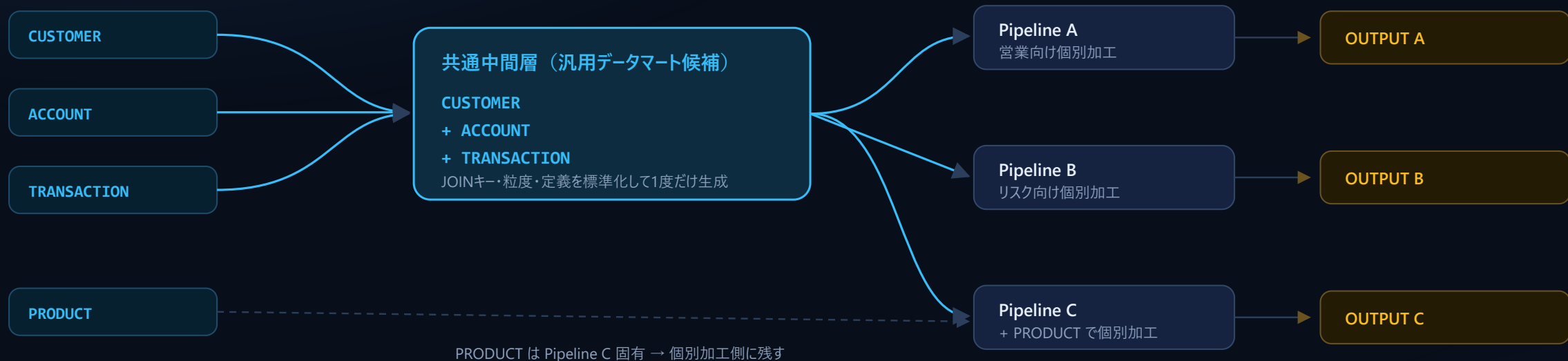
共通Input群



同じ CUSTOMER + ACCOUNT + TRANSACTION の結合結果が、Pipelineごとに別々に生成・保持されている。加工の重複、定義のばらつき、計算コストの重複が同時に発生する。

目指す姿：共通中間層を一度だけ作り、後続から再利用する

複数Pipelineに共通する連結部分を切り出す。その共通中間層そのものが汎用データマート候補になる。



最大の問いは「どこまでを共通中間層として作り、どこから先を個別Pipelineに任せるか」。この境界を、コードの利用実態から定量的に導く。

全体アプローチ：構造化 → グラフ化 → 連結分析 → 意味付け

最初から全件に詳細解析やLLM解析をかけない。安価で決定論的な処理で範囲を絞り、最後だけ解釈を足す。

01 構造化

ODM XML 約20,000件を Pipeline / Input / Output / Transform へ分解する

02 グラフ化

Table → Pipeline → Table の Data Usage Graph を Delta 上に構築

03 JOIN抽出

Pipeline内の直接JOIN関係を `pipeline_join_edge` として復元

04 連結集合生成

第1～第N連結の Connected Table Set を Pipeline 単位で生成

05 横断集計

Pipeline数・部門数・Output種類数・継続年数を集合ごとに集計

06 維持率分析

第N→第N+1で共通性がどれだけ残るかを計算し、汎用化境界を探す

07 Fingerprint

JOIN / Filter / 集計 / 粒度を正規化して Hash化し、加工ロジックを比較

08 Cluster化

完全一致でない類似Pipelineを束ねる（近似類似は候補絞り込み後に限定）

09 意味付け

必要な場合のみ LLM で業務概念・粒度・候補名を解釈（構造分析の後段のみ）

10 候補評価

汎用データマート候補をスコアリングし、人がレビューできる形で提示

検証段階の範囲

まずは第1～第4連結まで。複雑な総合スコアとLLM意味付けは結果を見てから追加する。

データモデル全体像：XML原本から候補一覧まで一気通貫

すべて Unity Catalog 上の Delta テーブルとして持つ。Graph DB を先に導入せず、SQL で分析できる形を正とする。



各テーブルの役割と汎用化への寄与 ①

テーブル・情報	保持する内容	汎用化への役割
odm_xml_raw	XML原本、file_hash、取込日時	再解析と証跡の起点。file_path + file_hash で霧等に取り込む
xml_field_mapping	XMLパス → 論理項目のマッピング定義	タグ構造を決め打ちせず、サンプル30件の観察結果を明文化する
pipeline_master	Pipeline、作成者、所属部門、作成日時	部門横断性と需要の継続性を評価する土台
pipeline_input	Pipelineが参照する Input Table	よく使われるInputを把握する
pipeline_output	Pipelineが生成する Output Table	繰り返し生成されているデータを把握する
pipeline_transform	JOIN / FILTER / AGGREGATION / GROUP_BY / CALCULATION	JOIN構造と加工内容を復元する。連結分析とFingerprintの原料
table_name_mapping	raw / normalized / canonical 名と根拠	表記ゆれの正規化と「意味的に別名の同一視」を明確に分離する
graph_node	Table / Pipeline の Node	データ利用構造の「点」
graph_edge	Table→Pipeline→Table の READS / WRITES	Fan-in、Fan-out、部門横断度、下流利用を分析する

各テーブルの役割と汎用化への寄与 ②

テーブル・情報	保持する内容	汎用化への役割
pipeline_join_edge	Pipeline内の直接JOIN関係、table_pair_key	どの2テーブルが直接つながるかを把握する
pipeline_connected_table_set	Pipeline内の第N連結テーブル集合、table_set_key	よくセットでJOINされるテーブル群を抽出する
connected_table_set_stats	集合ごとのPipeline数、部門数、Output数、継続年数	テーブル群の共通性を評価する
connected_table_set_parent_child	第N→第N+1の親子関係、retention_rate	連結を増やしたときの共通性の減衰を評価する
connected_table_set_candidate	共通中間層候補と評価指標、candidate_reason	汎用化境界の候補を提示する
table_usage_metrics	Fan-in / Fan-out / 部門数 / 継続年数	テーブル単位の利用実態を定量化する
pipeline_fingerprint	exact / structural の logic hash と各signature	名前が違って同じ加工をしているPipelineを発見する
pipeline_cluster	類似Pipeline群	完全一致ではない類似処理を束ねる
pipeline_semantic	業務概念・粒度・目的（LLM解釈）	技術的に似た処理へ業務的な意味を付与する
generic_mart_candidate	候補・スコア・推奨Tier・根拠	最終的な汎用データマート候補を整理する

Data Usage Graph : **Table** → **Pipeline** → **Table** で利用実態を持つ

「どのデータが、どこで、どれだけ利用・生成されているか」を見るための基本構造。Node/Edge を Delta テーブルとして保持する。



Fan-out

READS側。そのテーブルを読む distinct Pipeline数・部門数。**需要の広がり**を示す。

Fan-in

WRITES側。同じ出力テーブルを作る Pipeline 数。**重複生成**のシグナル。

部門横断度

同一部門100本より、**複数部門30本**の方が汎用化価値が高い可能性がある。独立指標として保持。

継続性

created_at から年を抽出し、単発の一時需要と**恒常需要**を分ける。

単一テーブルの頻度だけでは、**何を作るべきか**が決まらない

分かること

Fan-out / Fan-in を見れば頻出テーブルは分かる。

CUSTOMER が 1,500 Pipeline で使われる

しかし CUSTOMER は既にある。これは**作るべき新しい資産の姿**を教えてくれない。

本当に知りたいこと

複数Pipelineで繰り返し**セット利用**されるテーブル群。

CUSTOMER + ACCOUNT

CUSTOMER + ACCOUNT + TRANSACTION

CUSTOMER + ACCOUNT + TRANSACTION + PRODUCT

これを本資料では **Connected Table Set** と呼ぶ。

なぜ「セット」が重要か

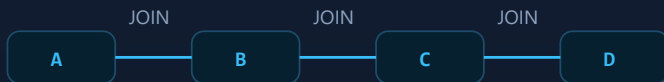
個々のテーブルは既に存在しているため、汎用化の対象は**それらを結合した結果**である。

繰り返し同じ組み合わせが結合されているということは、**その結合結果自体に恒常的な需要がある**ことの証拠になる。

第N連結 = N本のJOIN Edgeで連結される N+1 種類のテーブル集合

分析対象は単なるPATHではなく、同一Pipeline内でJOINにより接続された Connected Table Set である。

① 一直線のJOIN



第1連結 = A + B

第2連結 = A + B + C

第3連結 = A + B + C + D

② 一直線でない構造も対象



A + B + C は 2本のJOINで連結 → 第2連結

PATHではなく「連結された集合」として扱う

正規化ルール（必須）

テーブル集合の並び順で別候補にならないよう、`array_sort(table_set) → concat_ws('||', ...)` で `table_set_key` を生成する。

`CUSTOMER||ACCOUNT||TRANSACTION` と `TRANSACTION||CUSTOMER||ACCOUNT` は**必ず同一候補**として扱う。JOIN Edge も `table_pair_key` をソートして生成する。

Connected Table Set の生成：Pipeline内で1本ずつEdgeを足す

外部Graphライブラリは導入せず、Spark SQL / PySpark の反復処理で実装する。Pipeline間のCross Joinは行わない。

STEP 1

直接JOIN Edge

A-B
B-C
C-D

depth=1 / table_count=2



STEP 2

第2連結

A-B-C
B-C-D

depth=2 / table_count=3



STEP 3

第3連結

A-B-C-D

depth=3 / table_count=4



STEP 4

Pipeline横断集計

table_set_key ごとに
pipeline_count
department_count
output_table_count

各反復でやること

- 同じ pipeline_id 内に限定する
- 現在の table set に接続する Edge を1本選ぶ
- まだ含まれていないテーブルを1つ追加する
- array_sort で table set を正規化する
- 同一 pipeline_id + table_set_key を重複排除する

パフォーマンス・ガードレール

- 20,000 × 20,000 の Cross Join は**禁止**
- 連結生成は**Pipeline内で完結**させる
- MAX_CONNECTION_DEPTH = 4 を必ず設ける
- 利用Pipeline数が少ないSetは次Depthへ展開しない選択肢を持つ
- 外部Graphライブラリの新規導入・LLM利用はこの段階では行わない

JOIN復元の原則： `source_tables` に3テーブル以上あり、どのテーブル同士が直接JOINされているか判断できない場合、**全組み合わせを勝手にEdge化しない**。未解決として件数を記録する。

連結を深めると、共通性はどこで急落するか

テーブルを1つ足すたびに利用Pipeline数は減る。減り方の「形」を見ることで、共通化すべき範囲が見える。（下図は考え方を示す例）



読み取り

第1～第3連結までは多数のPipelineで共通。第4連結で急激に個別化している。

結論

CUSTOMER + ACCOUNT + TRANSACTION + PRODUCT までを共通中間層として切り出す境界候補と考える。

維持率（Retention）で 汎用化境界 を定量的に捉える

retention_rate

```
child_pipeline_count  
/ parent_pipeline_count
```

親子1組ごとの維持率。720 / 850 = 84.7%

parent_support_retention

```
child_pipeline_count  
/ MAX(parent_pipeline_count)
```

Parent候補が複数ある場合、最もPipeline数が多いParentを基準にした維持率。

max_child_retention

```
MAX(すべての子の  
retention_rate)
```

これが低いほど、その集合から先は個別化が進んでいる = 境界の可能性が高い。

Parent / Child の定義

Childのtable setから1テーブル除いた集合がParentに存在すれば親子候補。

Parent [A,B,C] → Child [A,B,C,D] なら added_table = D。

[A,B,C] には [A,B] [A,C] [B,C] という複数Parentがあり得るため、すべての親子関係を保存する。

汎用化境界候補の特徴

- 自身の Pipeline利用数が多い
- 複数部門から利用されている
- 複数Outputに利用されている
- ここまでは高い共通性を維持している
- さらにテーブルを足すと利用Pipeline数が急減する

暫定の候補判定ルール（パラメータ化して後から変更可能にする）： `pipeline_count >= 5 AND department_count >= 2 AND max_child_retention < 0.5 → candidate_flag = true`。この閾値は業務的な正解ではない。判定から漏れたデータも削除せず保持する。

汎用化候補を判断する主なシグナル

検証段階では、複雑な総合スコアより「人が判断できる事実」を並べることを優先する。

01 Pipeline数

何本のPipelineで同じテーブル群が使われているか。需要のボリューム。

02 部門数

何部門にまたがるか。部門横断性は件数と独立に評価する。

03 継続性

何年間同じ組み合わせが使われているか。単発需要と恒常需要を分ける。

04 Output種類数

同じ中間層から何種類のOutputが作られているか。再利用の広がり。

05 Parent Support Retention

1つ前の連結状態から、どの程度共通利用が維持されているか。

06 Child Retention

テーブルを追加した場合にどの程度残るか。急低下＝境界のシグナル。

07 加工ロジックの類似性

Fingerprintが同一・類似か。構造だけでなくロジックも共通か。

08 下流利用

生成結果がさらに多くのPipelineから参照されているか。

最終スコアリング（Generic Mart Score）の初期重み ※パラメータ化して変更可能にする

$0.30 \times \text{usage_frequency} + 0.25 \times \text{department_breadth} + 0.15 \times \text{continuity} + 0.15 \times \text{duplication} + 0.10 \times \text{downstream} + 0.05 \times \text{stability}$

各スコアは `percent_rank()` 等の順位ベースで0-100に正規化する。LLM confidence はスコアに混ぜず、意味付けの信頼度として別管理する。

「何をセットで使うか」と「どう加工するか」は別の軸

Connected Table Set

どのテーブル群がセットで使われているか

構造（データの組み合わせ）の共通性を見る。
今回の分析で最優先で正しく抽出する対象。

Pipeline Fingerprint

そのテーブル群をどう加工しているか

JOINキー・FILTER・集計・粒度の共通性を見る。
Connected Table Set の確度を後から補強する役割。

2段階で確度を上げる

STEP 1

CUSTOMER
+ ACCOUNT
+ TRANSACTION

が 500 Pipeline でセット利用される
→ 共通中間層候補

STEP 2

```
JOIN KEY = customer_id  
AGG = SUM(amount)  
GROUP BY = customer_id,  
          year_month
```

まで多数で一致 → より強い候補

共通するデータ構造の発見
+
共通する加工ロジックの発見
=
汎用化候補の確度向上

今回のスコープでは Fingerprint との統合スコアリングは実装対象外。ただし後から結合できるよう `pipeline_id` と `table_set_key` は必ず保持する。

Pipeline Fingerprint：名前ではなく**処理構造**で比較する

Pipelineが「何を入力し、どのように加工し、どの粒度で出力するか」という処理の特徴を、比較可能な形式で表したもの。

正規化してHash化する

```
INPUT=CUSTOMER,DEPOSIT,INVESTMENT
|JOIN=customer_id
|AGG=SUM(balance)
|GROUPBY=customer_id,year_month
```

表記順・大文字小文字・空白・alias差・関数名表記を正規化した上で SHA-256 でHash化する。

```
P001 → 8AF3897C...
P032 → 8AF3897C...
P182 → 8AF3897C...
```

同じHash = 実質的に同一の処理をしている可能性が高い。

A. Exact Fingerprint

literal値・条件値も含めて完全一致を見る。**実質的に同一ロジックの検出用。**

B. Structural Fingerprint

日付・数値・文字列などの literal を **<LITERAL>** にパラメータ化する。**条件値だけ違うテンプレート処理の検出用。**

ただし **カラム名・JOINキー・集約関数・GROUP BY粒度** など業務ロジックに影響する情報は保持する。

近似類似の扱い

20,000 × 20,000 の直積は禁止。input table数・共通input・output数・structural hash prefix 等で**bucketしてから** Jaccard 類似度を計算する。大規模化する場合は MinHashLSH を検討する。

Fingerprint → Cluster → Semantic : LLMは最後の解釈だけ

Fingerprintの完全一致だけでは、少し処理が異なる類似Pipelineを捉えられない。段階的に束ねてから意味を与える。

P001: CUSTOMER + DEPOSIT + INVESTMENT → SUM(balance)

P002: CUSTOMER + DEPOSIT + INVESTMENT + TRUST → SUM(balance)

Hashは異なるが、業務的にはどちらも「顧客金融資産」を作っている可能性がある。

決定論的

Fingerprint

exact / structural hash による完全・構造一致



決定論的

pipeline_cluster

bucket + 類似度で、完全一致でない類似処理を束ねる



LLM適用範囲

pipeline_semantic

業務概念・粒度・目的・候補名を解釈し
UNKNOWN を許容



決定論的

候補スコアリング

コード利用実態に基づく定量スコア。LLM
confidence とは分離

LLMに送るもの

candidate_id、input/output tables、JOIN/filter/集計/粒度
の概要、pipeline_count、department_count、
active_year_count、downstream数のみ。

LLMに送らないもの

Raw XML本文、業務データそのもの。20,000件全件も送らない
(MAX_LLM_CANDIDATES で上限管理)。

出力の扱い

指定JSONのみ返させ、根拠がない業務意味は UNKNOWN。
JSON parse失敗は記録し、自動再試行は最大2回。エンドポ
イント未設定でも構造分析は完走させ、LLM PhaseのみSKIPPED
として記録する。

Databricks上の実装フェーズ（Phase 0 – 11）

Genie Code の Agent mode に対し、コード生成だけでなくセル実行・エラー修正・品質検証までを指示する構成をとる。

P0 環境確認・サンプル調査

権限・Volume確認。XMLを30件だけ読み、tag構造を観察して `xml_field_mapping` を確定する

P1 XML Raw Ingestion

`binaryFile`で読み、file_hash付きで `odm_xml_raw` にMERGE

P2 Pipeline Metadata Parse

master / input / output / transform / parse_error に構造化

P3 Table名正規化

表記ゆれ正規化と意味的統合を分離。
`table_name_mapping`

P4 Data Usage Graph

`graph_node` / `graph_edge` をMERGEで構築し整合性を検証

P5 Graph分析指標

Fan-in / Fan-out / 部門横断 / 継続性 / 再帰Lineage

P6 Pipeline Fingerprint

exact / structural hash と重複クラスタ集計

P7 候補Feature作成

OUTPUT_TABLE型 と LOGIC_CLUSTER型の2種類で候補化

+ Connected Table Set分析

今回の追加分。join_edge → connected_table_set → stats → parent_child → candidate

P8 LLMによる意味付け

承認済み経路が利用可能な場合のみ。候補絞り込み後に限定

P9 候補スコアリング

`generic_mart_candidate` と recommendation_tier

P10/11 品質検証・実行サマリ

Acceptance Tests と `analysis_run_summary` (PASS / WARN / FAIL)

非交渉の実装ルール（ガードレール）

再現性・監査性・安全性を守るための前提。実装者・AIエージェントのどちらに対しても同じ制約をかける。

既存資産を壊さない

既存テーブルへの DROP / TRUNCATE / DELETE / 破壊的 OVERWRITE は禁止。書き込みは専用の `TARGET_CATALOG.TARGET_SCHEMA` に限定する。同名テーブルがある場合はスキーマと行数を確認してから MERGE する。

推測で補完しない

XMLに存在しない業務定義・部門・テーブルの意味を推測しない。不明値は NULL または UNKNOWN として保持する。pipeline_id が欠落しても業務IDを生成せず `TECH:<file_hash>` とする。

仕様不明のまま全量Parseしない

必ずサンプル観察とマッピング確定を先に行う。XML構造が複数パターンある場合は `parser_version` / `xml_pattern` を記録し、パターン別に処理する。

計算量を爆発させない

Pipeline間の 20,000 × 20,000 直積を作らない。まず Hash / Bucket / 入力集合で候補を絞る。連結生成は Pipeline 内で完結させ、各Depthで重複排除する。

LLMは最小限・後段のみ

構文・構造解析は Spark / Python で決定論的に行う。20,000件全件を LLM へ送らない。Raw XML本文や業務データを外部へ送らない。APIキー・Secretをノートブックへ平文で書かない。

冪等性と検証を必須にする

同一ファイルを再実行しても重複行が増えないよう `file_path + file_hash` や `pipeline_id` をキーにする。各フェーズ終了時に検証SQLを実行し、行数・NULL率・重複・エラー件数を表示する。エラーは握りつぶさず原因と修正内容を残す。

実行前に決めるパラメータ

プレースホルダが未置換の場合、既存本番スキーマへ書き込まない。閾値はすべてNotebook上で変更できるようにする。

環境パラメータ

パラメータ	意味・既定値
TARGET_CATALOG	解析用テーブルを作成する Unity Catalog Catalog 必須
TARGET_SCHEMA	既定 odm_analysis。専用Schemaを推奨 必須
XML_VOLUME_PATH	20,000件のODM XMLが存在するVolumeパス 必須
SOURCE_SYSTEM	既定 ODM。原則そのまま
AZURE_OPENAI_ENDPOINT	承認済み Model Serving / External Model endpoint 名。未設定ならLLM PhaseのみSKIP
MAX_LLM_CANDIDATES	LLMへ送る候補上限。既定 1,000
SAMPLE_XML_COUNT	構造観察用サンプル件数。既定 30

連結分析パラメータ

パラメータ	意味・既定値
MAX_CONNECTION_DEPTH	探索する最大連結深度。初回検証は 4
MIN_PIPELINE_COUNT	次Depthへ展開する最小Pipeline数。既定 2
MIN_DEPARTMENT_COUNT	集計対象の最小部門数。既定 1
MIN_CANDIDATE_PIPELINES	候補判定の最小Pipeline数。既定 5
MIN_CANDIDATE_DEPARTMENTS	候補判定の最小部門数。既定 2
MAX_CHILD_RETENTION_THRESHOLD	この値未満なら境界候補。既定 0.5

初回検証は**第1～第4連結まで**。実データ件数・処理時間・候補数を確認してから拡張できる設計にする。候補判定から漏れたデータも**削除せず保持**する。

品質検証：実装後に必ず確認すること

A. Raw

XML発見件数 / distinct file_path / 空ファイル件数 / hash重複件数

B. Parse

distinct pipeline_id / success・error件数と率 / department取得率 / created_at取得率 / input・output取得率

C. Graph

node重複 = 0 / edge重複 = 0 / orphan edge = 0 / READSはTABLE→PIPELINEのみ / WRITESはPIPELINE→TABLEのみ

D. サンプル照合

XML 30件を原文とParse結果で突合し、一致 / 不一致 / 確認不能を集計する

JOIN・Connected Set の品質確認

- JOIN Transform総数 / Edgeへ変換できた件数 / できなかった件数
- 変換不能理由 Top10 を必ず提示する
- 第1～第4連結ごとの候補数を表示し、**候補数が異常に爆発していないか**確認する
- Pipelineを最低5件抽出し、元のJOIN → pipeline_join_edge → connected_table_set が一致することを目視で確認する

最終提示物（Notebookに日本語で出力）

- JOIN解析結果（Transform数 / 抽出成功 / 失敗）
- 第N連結別の候補数
- 各DepthのTop20（table_set / pipeline_count / department_count / active_year_count / output_table_count）
- 維持率の具体例を3～5件、連結の流れが見える形で
- 汎用化境界候補 Top20（parent_support_retention / max_child_retention / candidate_reason 付き）
- 品質上の懸念と、次に人間がレビューすべき論点

`analysis_run_summary.quality_status` の目安：orphan edge > 0 または node/edge重複 > 0 → FAIL。parse error率が閾値以上 → WARN（閾値と実値をnotesに記録）。サンプル照合に重大不一致 → WARN / FAIL。

この分析で答えたい問い

実行結果を使って、最終的に以下へ回答できる状態にする。

Q1 複数Pipelineで頻繁にセット利用されるテーブル群は何か

Q2 第1～第4連結で、それぞれどのテーブル群が多いか

Q3 連結数を増やしても高い共通利用率を維持するテーブル群は何か

Q4 どの連結段階からPipeline利用数・部門数が大きく低下するか

Q5 「ここまでを共通中間層として持ち、それ以降を個別Pipelineで加工する」境界候補はどこか

Q6 同じテーブル群が複数部門・複数Output用途に利用されている例は何か

Q7 そのテーブル群に対する JOIN・Aggregation・粒度まで共通しているか

ゴールは「最もよく使われるテーブル」を探すことではない。

複数の既存Pipelineで繰り返し構築されている“連結済みの中間データ構造”を発見し、共通中間層・汎用データマート候補として検討できる状態にすること。

まとめ：6つの視点と、超えてはいけない一線

Graph

どこで・どれだけ使われているか

Connected Table Set

どのテーブル群がセットで使われているか

第N連結 / Retention

どこまでを共通中間層にすべきか

Fingerprint

そのテーブル群をどのように加工しているか

Cluster

どのPipeline同士が似ているか

Semantic

業務的に何を作ろうとしているか

よくセットでJOINされる + 複数Pipelineで使われる + 複数部門で使われる + 複数Outputに再利用される + 第N連結まで共通性が維持される + それ以上の連結で個別化が進む + 加工ロジックも共通・類似している

— そのような中間データほど、個別Pipeline内で毎回作るのではなく、共通データ資産として一度整備し再利用する価値が高い。

注意事項：コード上の頻出は、業務定義として正しいことを意味しない

Connected Table Set が頻出していても、JOINキー・JOIN Type・Filter条件・粒度・対象商品/顧客範囲が異なる可能性がある。

コード分析 → Connected Table Setの発見 → 汎用化境界候補の抽出 → Fingerprintによるロジック確認 → **業務定義の確認・統一** → 正式な汎用データマート

コード解析結果は「共通データ需要を発見するための証拠」であり、そのまま正解データ定義としないことが重要である。